HOME RENTAL PLATFORM

Owner-Based SaaS Product Structure & Functional Requirements

A product blueprint for property listings, tenant onboarding, rent collection, and property operations

Version 1.0 | October 2026

1. Executive Summary

The application is an owner-based, subscription SaaS platform for rental-property management. The platform administrator creates and manages owner accounts. Each owner receives a private management dashboard and an owner-branded public website with its own shareable link or connected custom domain. The homes and rooms published by that owner appear on that owner's website, rather than being mixed into one mandatory shared marketplace. Renters browse that owner's listings, contact the owner through email, WhatsApp, or in-app chat, apply for a home, complete the rental process, upload requested verification documents, pay rent through supported methods, and raise requests during their tenancy. Owners manage multiple properties, multiple units, shared occupancy, monthly rent, tenant documents, requests, and analytics from their dashboard. Owners pay a recurring subscription to use the service; the platform administrator manages plans, billing status, limits, domains, and account provisioning.

The platform should distinguish a property (for example, a building or house) from a rentable unit (for example, an apartment, room, or bed). This matters when one owner has several houses or when several tenants share a single property.

Owner-Based SaaS Model (Core Product Decision)

This product is not primarily a single public rental marketplace. It is a multi-tenant SaaS platform where each landlord or property owner operates their own rental website, powered by the same central application. The platform operator/admin controls account creation and subscription administration; each owner controls only their own brand, listings, tenants, payment settings, and property operations.

Platform administrator workflow

- Admin creates an owner account and enters the owner's name, email, initial plan, subscription status, allowed property/unit limits, and onboarding status.
- The system sends the owner a secure invitation to activate the account, set a password, verify email, and complete profile/business details.
- The admin assigns or confirms the owner's website address: for example, ownername.platformdomain.com, or a custom domain such as rentals.ownerbrand.com.
- The admin can activate, suspend, upgrade, downgrade, or cancel an owner account according to the subscription and platform policies.
- Admin can view subscription/payment status and technical health, but should not routinely access private tenant documents or owner communications without an authorised support reason and audit log.

Owner workspace and branded website

- Each owner signs in to a private dashboard to manage properties, units, listings, tenant applications, active tenancies, rent, documents, messages, requests, and analytics.
- Each owner has a public mini-site showing only that owner's published listings, logo/brand name, approved contact details, description, and relevant rental policies.
- The owner can share their unique URL directly with prospective tenants. A visitor using that URL should see the relevant owner's homes only by default.
- A central platform homepage may explain the SaaS product and allow owners to sign in, but it is separate from each owner's public rental website.
- An optional platform-wide directory/search can be added later, but it must not be required for the owner-based model to work.

Subscription and plan management

- Owners subscribe to use the platform, with monthly and/or annual billing. Plan options may limit active properties, rentable units, staff accounts, storage, custom domains, or advanced reports.
- Keep subscription billing separate from tenant rent collection. The owner's subscription is paid to the platform operator; rent is paid by tenants to the owner or the owner's configured payment recipient.

- Track plan, billing interval, subscription status, trial start/end, current period start/end, payment provider customer/subscription IDs, invoice history, and cancellation date.
- Define access rules for past-due subscriptions: notify first, provide a grace period, then restrict selected owner functions if needed. Do not silently delete listings, tenancy records, documents, or payment history.
- When a subscription is cancelled or expires, apply a clear policy for the public owner site (for example, unpublished or read-only) while preserving legally or operationally required records.
- Support plan changes, failed subscription payments, renewal reminders, invoices/receipts, tax handling, and admin-issued comped or manually managed plans if needed.

Custom domains and owner URL routing

- MVP: give every owner a unique platform subdomain, such as ownername.platformdomain.com. This is easier to provision and validate than custom domains.
- Later or on eligible plans: allow owners to connect a domain they own. Provide DNS instructions, domain verification, TLS/HTTPS provisioning, and clear pending/verified/error states.
- Route each incoming hostname to the correct owner workspace/site using a verified domain-to-owner mapping. Never trust a URL parameter alone to determine tenant identity.
- Prevent one domain from being attached to multiple owners; validate domain ownership and safely handle domain changes, expiry, and removal.
- Public pages should be SEO-friendly and mobile responsive. Owner dashboard routes must remain private and must not be exposed merely because the public site uses the owner's domain.

Multi-tenant data isolation

- Every owner-owned record must carry an owner/tenant scope or be reachable only through a parent record that is securely scoped to that owner.
- Every backend request must verify that the signed-in user is allowed to access the requested owner, property, unit, listing, tenant, payment, document, or request.
- Use automated tests to confirm Owner A cannot read or change Owner B's data by changing IDs, URLs, filters, or API payloads.
- For custom domains, determine the public site owner from a verified hostname mapping, but determine private dashboard access from authenticated identity and permissions.
- Use audit logs for admin impersonation/support access, subscription changes, domain changes, payment adjustments, and sensitive document review.

Important financial separation

- Platform subscription invoices are issued by the SaaS operator to property owners.
- Tenant rent charges and receipts belong to the relevant owner/tenancy and must not be counted as SaaS subscription revenue.
- Owner payout/bank settings are configured separately for each owner. The system must not route rent from one owner to another owner.
- If the platform ever collects tenant rent and distributes it to owners, assess payment-provider marketplace capabilities and legal/regulatory obligations before implementation.

2. Product Goals

- Give owners one place to manage all properties, listings, tenants, rent, documents, and requests.
- Help renters search and compare homes, ask questions, apply, and manage their tenancy.
- Support shared accommodation where multiple tenants can be associated with one property or unit.
- Make rent status transparent: due, paid, partially paid, overdue, disputed, or waived.
- Keep sensitive identity and tenancy documents private, access-controlled, and auditable.
- Provide clear operational analytics without presenting unconfirmed payment attempts as completed payments.

3. User Roles & Permissions

Role	Primary capabilities	Access boundaries
Visitor	Browse published listings, search/filter, view listing details.	Cannot view private owner/tenant data or documents.
Renter / Applicant	Save homes, contact owners, submit an application, upload requested documents, review lease/rent details, pay rent, chat, raise requests.	Can access only their own profile, applications, tenancy, payments, documents, and requests.
Property Owner / Landlord	Manage own branded rental website, properties/units, listings, applications, tenants, rent, documents, conversations, requests, analytics, and owner-specific payment settings.	Can access only their own tenant workspace and records; plan limits apply.
Property Manager (optional)	Perform delegated owner tasks such as listing updates, tenant communication, and request handling.	Only assigned properties and permissions.
Platform Admin	Create/provision owner accounts, assign plans, manage subscriptions, domains, account status, platform support, moderation, and integration health.	Privileged actions require audit logs; access to tenant documents is restricted and purpose-limited.

4. Application Areas / Main Navigation

Renter application

- Discover Homes
- Saved Homes
- My Applications
- Messages
- My Tenancy
- Rent & Payments
- Documents / Verification
- Requests & Maintenance
- Notifications
- Profile & Settings

Owner dashboard

- Overview
- My Properties
- Listings
- Applications / Leads
- Rented Properties
- Tenants
- Rent & Payments
- Documents
- Messages
- Maintenance & Requests
- Reports / Analytics
- Owner Profile & Payout Settings

Admin console

- User and owner management
- Listing moderation and reports
- Payment / webhook monitoring

- Dispute and support queue
- Audit logs
- Platform configuration

5. Core Domain Structure

Entity	Meaning	Example
Owner	Account that owns or manages properties.	One landlord managing 4 houses.
Property	Physical address/building.	A 3-floor house in Berlin.
Unit	Independently rentable space within a property.	Entire house, apartment 2B, or Room 3.
Listing	Public advertisement for a unit, with price, photos, rules, availability, and description.	Room in a shared apartment, available from 1 November.
Application / Inquiry	A renter's expression of interest and application status.	Renter asks to view a room and submits details.
Tenancy	Confirmed rental relationship for a unit or property over a date range.	Three tenants sharing one house under separate agreements.
Tenant membership	Connects a tenant to a tenancy, including dates and status.	Tenant A active; Tenant B moving in next month.
Rent charge	An amount due for a particular tenant or tenancy period.	November rent due on the 3rd.
Payment	A payment attempt and its verified final status.	€850 rent paid successfully.
Document	Uploaded file with type, owner, access rules, review status, and expiry where relevant.	Proof of identity or proof of income.
Request / Ticket	Tenant issue or service request with messages, priority, and status.	Heating repair requested.
Conversation	Communication thread linked to a listing, application, or tenancy.	Owner and applicant discuss a viewing.

Important modelling rule: do not treat "house" and "tenant" as a simple one-to-one relationship. A property can contain multiple units, and a unit or shared tenancy may have multiple tenants. Store the actual rental arrangement in a tenancy record and connect tenants through a tenant-membership table. This avoids problems when someone moves in or out.

6. End-to-End User Workflows

6.1 Owner publishes a home

1. Owner creates an account and completes required owner verification.
2. Owner adds a property with address, property type, amenities, photos, rules, and ownership/management details.
3. Owner creates one or more rentable units and sets rent, deposit, utilities, availability date, occupancy limits, and rental terms.
4. Owner previews and publishes a listing. The listing becomes searchable only when it passes required checks.
5. Owner can pause, edit, mark unavailable, or archive a listing; changes are reflected in renter search.

6.2 Renter discovers and contacts owner

6. Renter searches by location, budget, move-in date, property type, rooms, furnished status, and other filters.
7. Renter opens a listing and reviews photos, price breakdown, amenities, rules, availability, and owner profile.
8. Renter can save the listing and contact the owner by in-app chat, email, or WhatsApp.
9. Contact actions should preserve context by linking the conversation or inquiry to the specific listing.
10. If the renter proceeds, they submit an application or viewing request; the owner can accept, reject, or request more information.

6.3 Booking, agreement, and move-in

11. Owner and renter agree on the unit, move-in date, rent, deposit, payment schedule, and tenancy terms.

12. The system creates a draft tenancy or booking record; it must not mark the unit as rented solely because someone clicked “I am interested”.
13. Required documents are requested and uploaded through secure, private links.
14. Owner reviews the application/documents and records the decision. Where appropriate, both parties sign or acknowledge the rental agreement.
15. Any deposit or initial payment is handled through a supported payment flow with clear receipts and refund/cancellation rules.
16. Only after the configured confirmation conditions are met does the tenancy become active and the unit move from Available/Reserved to Rented.

6.4 Owner views rented property and tenants

17. Owner opens Rented Properties and sees active, upcoming, and ended tenancies.
18. Selecting a property shows its units and occupancy; selecting a unit shows all active tenants linked to it.
19. Each tenant profile shows permitted contact details, tenancy dates, rent responsibility, document review status, payment history, and open requests.
20. Owner can add an approved co-tenant, record a move-out, or update occupancy, with a timestamped history.
21. The system must prevent accidental deletion of historical tenancy, rent, or payment records.

6.5 Monthly rent collection

22. A rent schedule defines amount, currency, due date, frequency, grace period if any, and who is responsible for payment.
23. Before rent is due, the app sends reminders to the responsible tenant(s).
24. Tenant opens the charge, sees the amount and payee/owner, and selects a supported payment method.
25. The payment provider processes the transaction or the tenant follows the displayed bank-transfer instructions.
26. The backend confirms status from a trusted provider webhook or reconciled bank/payment record; the frontend must not mark a payment paid based only on a redirect or button click.
27. The rent dashboard updates paid, unpaid, partial, overdue, failed, or disputed amounts and issues a receipt where applicable.
28. Owner can record an offline payment with date, amount, reference, and evidence, clearly labelled as manually recorded.

6.6 Tenant raises a request

29. Tenant creates a request with category, description, optional photos, urgency, and preferred access times.
30. Owner/manager receives a notification and assigns a status: New, Acknowledged, In Progress, Waiting for Tenant, Resolved, or Closed.
31. Tenant and owner exchange messages and attachments within the request thread.
32. The system records timestamps, status changes, assignee, and resolution notes.
33. Tenant can reopen a request if the issue is not resolved, according to configurable rules.

7. Functional Requirements by Module

7.1 Authentication & profiles

- Email/password or passwordless sign-in; email verification; password reset; optional phone OTP and social sign-in.
- Role-aware onboarding for renter, owner, and optional property manager.
- Profile settings, notification preferences, session management, account deletion/request process.
- Owner verification status and renter identity/document verification status should be separate fields.

7.2 Property and listing management

- Support multiple properties per owner and multiple rentable units per property.
- Listing fields: title, description, address/location, unit type, rent, deposit, utilities, availability, occupancy, furnishing, amenities, rules, photos, and contact preferences.

- Listing statuses: Draft, Pending Review, Published, Paused, Reserved, Rented, Archived, Rejected.
- Photo ordering, image validation, duplicate listing checks, listing report/flagging, and edit history.
- Search and filtering by map/location, price, move-in date, unit type, rooms, amenities, and availability.

7.3 Applications and communications

- Inquiry, viewing request, application submission, owner decision, follow-up questions, and application status timeline.
- In-app chat should be the durable conversation record; email and WhatsApp can be optional outbound contact channels.
- Use verified, consent-based contact sharing and anti-spam/rate limits.
- Messages should support text, timestamps, read state, and controlled attachment types.

7.4 Tenancy and occupancy

- Tenancy start/end dates, agreement reference, rent terms, deposit, payment responsibility, occupancy limit, and status.
- Allow multiple tenants per unit/tenancy, including separate rent shares where needed.
- Support tenant invitation/acceptance, co-tenant changes, move-in, move-out, renewal, and termination records.
- Do not make a unit available until occupancy and tenancy rules permit it.

7.5 Verification documents

- Allow an owner to request specific document types; renter uploads from device and sees upload/review status.
- Document states: Requested, Uploaded, Under Review, Accepted, Rejected, Expired, Re-upload Required.
- Use private object storage, short-lived signed access URLs, file-size/type limits, malware scanning where available, and encryption.
- Access should be limited to the relevant tenant, authorised owner/manager, and explicitly authorised support/admin roles.
- Collect only documents needed for the stated purpose; define retention and deletion rules before launch.

7.6 Rent, payments, and receipts

- Recurring rent schedule generation and charge ledger, with due date, amount, currency, tenant responsibility, and status.
- Support full and partial payments, failed attempts, refunds, reversals, disputes, manual/offline payments, and payment references.
- Provide owner payout/bank details management, verification, and clear separation between payment collection and payout.
- Support downloadable receipts and a complete audit trail for changes to charges and payment records.
- Wise or a bank transfer must be integrated only through a supported provider/API or a clearly explained manual transfer/reconciliation flow; do not assume every bank supports direct in-app debit.

7.7 Requests and maintenance

- Request categories such as repair, plumbing, heating, electricity, cleaning, access, noise, billing, and other.
- Priority, attachments, comments, assignee, target date, status history, and resolution notes.
- Notifications on creation, assignment, status changes, and replies.
- Owner dashboard for open, overdue, high-priority, and resolved requests.

7.8 Notifications

- In-app notifications and email for application updates, new messages, document requests, rent reminders, payment confirmations, overdue rent, and maintenance updates.
- Optional push notifications for mobile apps.
- User-configurable preferences, quiet hours where appropriate, and notification delivery logs.

7.9 Analytics and reporting

- Owner overview: properties, available units, rented units, occupancy rate, active tenants, rent due this month, amount collected, outstanding rent, overdue charges, and open requests.
- Rent report: paid vs unpaid by month, tenant, property, unit, and due date.
- Property performance: occupancy timeline, vacancy days, income collected, and upcoming availability.
- Exportable CSV/PDF reports with access control and clear reporting period.
- Analytics must use verified payment statuses and should display when figures were last updated.

7.10 Admin and support

- Review reported listings, suspend abusive accounts, resolve support tickets, and inspect integration/webhook health.
- View system audit logs and security events without granting unnecessary access to tenant documents.
- Admin actions such as account suspension, listing removal, and payment corrections require reason and audit trail.

8. Key Screens and What Each Screen Shows

Screen	Main content / actions
Home / Search	Search bar, location, filters, listing cards, map/list toggle, saved homes.
Listing details	Photos, description, price breakdown, amenities, rules, availability, owner details, report listing, contact buttons.
Owner overview	Portfolio summary, rent status, occupancy, alerts, recent activity, requests requiring attention.
Property details (owner)	Property profile, units, listing status, occupancy, tenants, documents, rent summary, requests.
Rented unit / tenants	Active and upcoming tenants, tenancy dates, rent responsibilities, document status, payment history, open requests.
Tenant tenancy page	Address/unit, owner contact, agreement details, next rent due, payment history, documents, requests.
Rent checkout / transfer	Charge amount, currency, recipient, method, fees if any, payment reference, status, help and receipt.
Documents	Requested document type, upload action, review status, feedback, expiry/re-upload requirement.
Messages	Conversation list grouped by listing/application/tenancy, message history, attachment support.
Requests	Ticket list, category, status, priority, details, comments, attachments, resolution history.
Analytics	Filters for date/property/unit; collection totals, outstanding amounts, occupancy, payment and request trends.

9. Suggested Data Model

Table / collection	Important fields (illustrative)
users	id, email, phone, display_name, role, account_status, email_verified_at, created_at
owner_profiles	user_id, verification_status, payout_profile_id, business_or_individual_type
renter_profiles	user_id, phone, preferences, verification_status
properties	id, owner_id, name, address, geo_location, property_type, status
units	id, property_id, unit_name, unit_type, bedrooms, bathrooms, occupancy_limit, status
listings	id, unit_id, title, description, rent_amount, currency, deposit_amount, available_from, status, published_at
listing_media	id, listing_id, storage_key, media_type, sort_order
applications	id, listing_id, renter_id, status, desired_move_in, submitted_at, decision_at
conversations	id, context_type, context_id, created_at
conversation_members	conversation_id, user_id, last_read_at

messages	id, conversation_id, sender_id, body, attachment_key, created_at
tenancies	id, unit_id, start_date, end_date, status, rent_amount, currency, deposit_amount, agreement_reference
tenancy_members	id, tenancy_id, tenant_user_id, rent_share_amount, start_date, end_date, membership_status
document_requests	id, tenancy_id/application_id, tenant_user_id, document_type, required, due_date, status
documents	id, owner_user_id, related_tenancy_id, document_type, storage_key, review_status, expires_at, created_at
rent_charges	id, tenancy_id, responsible_tenant_id, period_start, period_end, due_date, amount, currency, status
payments	id, rent_charge_id, payer_id, provider, provider_reference, amount, currency, status, paid_at
payment_events	id, payment_id, event_type, provider_event_id, payload_hash, received_at
payout_accounts	id, owner_id, provider, external_account_reference, verification_status
requests	id, tenancy_id, created_by, category, priority, title, description, status, assignee_id, created_at
request_comments	id, request_id, author_id, message, attachment_key, created_at
notifications	id, user_id, type, channel, status, sent_at, read_at
audit_logs	id, actor_user_id, action, entity_type, entity_id, reason, created_at
owner_sites	id, owner_id, site_name, logo_key, brand_settings, public_slug, publication_status
owner_domains	id, owner_id, hostname, verification_status, tls_status, verified_at, last_error
subscription_plans	id, name, billing_interval, price, currency, feature_limits, active
owner_subscriptions	id, owner_id, plan_id, provider_customer_id, provider_subscription_id, status, trial_end, current_period_end, cancel_at_period_end
subscription_invoices	id, owner_subscription_id, provider_invoice_id, amount, currency, status, due_at, paid_at

Use database constraints and transactions for important state changes. A payment should reference a rent charge; a tenancy should reference a unit; and tenant membership should reference a tenancy. Avoid storing raw bank credentials or full payment-card details in the application database.

10. Business Rules and Statuses

Area	Suggested statuses / rule
Listing	Draft → Pending Review (if moderation is enabled) → Published → Reserved/Rented → Archived; Paused and Rejected are alternative states.
Application	New → Viewing Requested / In Review → More Information Needed → Accepted or Rejected → Withdrawn/Expired.
Tenancy	Draft → Awaiting Documents/Agreement/Payment → Upcoming → Active → Ended; Cancelled is an explicit terminal state.
Rent charge	Upcoming → Due → Partially Paid / Paid; Overdue is derived from due date and outstanding balance; Disputed/Waived require a reason.
Payment	Initiated → Pending → Succeeded / Failed / Cancelled / Refunded / Reversed; final status comes from trusted provider confirmation or reconciliation.
Document	Requested → Uploaded → Under Review → Accepted or Rejected; expired documents can become Re-upload Required.
Maintenance request	New → Acknowledged → In Progress → Waiting for Tenant → Resolved → Closed; reopening may be allowed.
Occupancy	The unit can be marked Rented only when the configured tenancy activation conditions are satisfied; never infer it from an inquiry alone.

11. Payment Architecture: Wise and Bank Accounts

Payment design should be validated before implementation because “pay rent from Wise or any bank account” can mean different things. The app can show an owner’s verified receiving details and let tenants make a bank transfer, or it can use a payment provider that supports the required payment method, market, currency, and payout model. Wise availability and API permissions depend on the account type, country, and product; do not assume a Wise account can be debited directly by the app.

- MVP option: display verified owner bank-transfer details, create a unique payment reference for each rent charge, let the tenant mark a transfer as sent, and reconcile it against bank/provider data or owner confirmation. Label this as “awaiting confirmation” until verified.
- Integrated option: use a payment service provider that supports the required country, tenant payment methods, marketplace/landlord payouts, refunds, and webhook events.
- Store provider tokens/references rather than online banking passwords. Never ask tenants to submit bank login credentials.
- Verify webhook signatures, make webhook processing idempotent, and preserve raw event metadata safely for reconciliation.
- Define who legally receives funds, who pays provider fees, how refunds/deposits work, and how disputes are handled before launch.
- Show separate statuses for rent charge, payment attempt, and owner payout. A successful tenant payment does not necessarily mean the owner payout has settled.

12. Security, Privacy, and Compliance Considerations

- Use role-based access control and property/tenancy-level authorization on every API request.
- Encrypt traffic with HTTPS and encrypt sensitive files and secrets at rest; use a secrets manager or environment secrets, not committed credentials.
- Store verification files in private object storage and provide short-lived signed URLs only after permission checks.
- Log document views/downloads and sensitive owner/admin actions; avoid putting personal document contents into ordinary application logs.
- Apply file size/type limits, malware scanning where feasible, rate limiting, input validation, secure session handling, and backup/restore testing.
- Define retention, deletion, consent, access, and correction workflows for personal data. For a Germany/EU launch, obtain appropriate legal advice on GDPR, rental practices, payment regulation, consumer information, and required disclosures.
- Do not collect or expose more personal information than needed for a clear rental purpose. Avoid displaying one tenant’s private documents to other co-tenants.
- Have clear policies for deposits, late fees, cancellations, disputes, maintenance emergencies, and owner/tenant responsibilities.

13. Recommended Technical Architecture

A practical implementation is a responsive web application first, with a mobile app or PWA added when the core workflows are stable. The backend should remain the single source of truth for web and mobile clients.

Layer	Suggested implementation	Notes
Web frontend	Next.js + TypeScript	Public listing website, renter experience, owner dashboard, responsive layout.
Mobile (later / optional)	Flutter or responsive PWA	Reuse the same backend APIs; prioritise push notifications if native apps are needed.
Backend API	Node.js + Express.js + TypeScript	Authentication, listings, tenancy, payments, documents, requests, analytics, integrations.
Database	PostgreSQL (for new relational design) or	Rental relationships, rent ledger, tenancy

	retain existing MongoDB if already implemented	memberships, and payment reconciliation benefit from relational constraints and transactions. Choose one primary database; avoid dual-write complexity.
File storage	Private AWS S3 or equivalent object storage	Listing photos and sensitive documents should use separate access policies/buckets or prefixes.
Email	Transactional email provider / NodeMailer integration	Verification, application updates, rent reminders, receipts, and request notifications.
Payments	Provider adapter + verified webhooks / bank-transfer reconciliation	Keep provider-specific logic isolated from rent-charge business logic.
Background jobs	Queue/worker for reminders and recurring rent charges	Prevent duplicate charges; use idempotency keys and job monitoring.
Deployment	Docker, HTTPS reverse proxy, monitoring, backups	Separate development/staging/production, centralise logs, alert on payment/job failures.

Database recommendation: if the current rental application already has a working MongoDB Community deployment, it can be retained for an initial release, but model payment and tenancy invariants carefully. If the project is still early, PostgreSQL is a strong default for the connected entities and financial ledger. Make the choice before building many modules.

14. API Surface (Illustrative)

Area	Example endpoints
Auth	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout
Listings	GET /listings, GET /listings/:id, POST /owner/listings, PATCH /owner/listings/:id, POST /owner/listings/:id/publish
Properties / units	GET /owner/properties, POST /owner/properties, POST /owner/properties/:id/units, PATCH /owner/units/:id
Applications	POST /listings/:id/applications, GET /owner/applications, PATCH /owner/applications/:id/status
Tenancy	GET /owner/tenancies, GET /tenancies/:id, POST /owner/tenancies, POST /tenancies/:id/invite-tenant, PATCH /tenancies/:id/members/:memberId
Documents	POST /documents/upload-url, POST /documents/confirm-upload, GET /tenancies/:id/documents, PATCH /owner/documents/:id/review
Rent / payments	GET /tenancies/:id/rent-charges, POST /rent-charges/:id/payment-intent, POST /payments/webhooks/:provider, GET /owner/reports/rent
Requests	POST /tenancies/:id/requests, GET /tenancies/:id/requests, PATCH /requests/:id/status, POST /requests/:id/comments
Messages	GET /conversations, POST /conversations, GET /conversations/:id/messages, POST /conversations/:id/messages
Analytics	GET /owner/dashboard/summary, GET /owner/analytics/rent, GET /owner/analytics/occupancy
Platform admin / owners	POST /admin/owners, PATCH /admin/owners/:id/status, POST /admin/owners/:id/invite
Owner sites / domains	GET /owner/site-settings, PATCH /owner/site-settings, POST /owner/domains, POST /owner/domains/:id/verify
Subscriptions	GET /owner/subscription, POST /owner/subscription/checkout, POST /billing/webhooks/:provider, GET /owner/invoices

These are example route names, not a final API contract. Every endpoint needs authentication, authorization, validation, pagination where appropriate, and consistent error responses.

15. Owner Analytics: Recommended Metrics

Metric	Definition / display rule
Total properties	Count of active properties managed by the owner.

Available units	Units published and genuinely available for the selected period.
Occupancy rate	Occupied rentable units divided by total rentable units; state the exact definition and period.
Rent expected	Sum of rent charges due in the selected period, excluding cancelled charges.
Rent collected	Sum of confirmed successful payments allocated to rent charges in the selected period.
Outstanding rent	Total due charges minus confirmed allocations, with refunds/reversals handled correctly.
Overdue tenants	Tenants with an outstanding balance after the due date, based on configured rules.
Collection rate	Confirmed rent collected divided by rent expected for the selected period.
Upcoming due dates	Charges due in the next configured window.
Open requests	Count of non-closed maintenance/service tickets, with high-priority and overdue subsets.

16. MVP Scope vs Later Releases

Release	Include
MVP — core rental operations	Sign-up/login; owner and renter profiles; properties/units/listings; search and listing details; contact via email/WhatsApp/in-app chat; application tracking; tenancy creation; multi-tenant occupancy; secure document upload/review; monthly rent ledger; bank-transfer instructions and manual reconciliation or one selected provider; owner rent dashboard; tenant requests; email notifications; admin moderation; audit logs.
Release 2 — automation	Automated recurring charge generation; integrated provider checkout/payout where supported; reminders and overdue workflows; tenant invitations/co-tenant management; downloadable receipts/reports; richer maintenance assignment and reporting.
Release 3 — expansion	Native mobile apps/push notifications; e-signature integration; accounting exports; multilingual support; advanced analytics; verified property-manager delegation; integrations selected based on actual customer demand.

Keep the MVP focused. Payment integrations, legal rental workflows, and document handling carry more risk than basic listing screens, so validate these early with a small end-to-end prototype.

17. Non-Functional Requirements

- Responsive UI for desktop, tablet, and mobile; accessible forms and clear empty/error/loading states.
- Secure authorization enforced server-side; frontend visibility rules are not security controls.
- Reliable rent ledger with immutable transaction history and reconciliation capability.
- Idempotent payment/webhook processing and duplicate-safe recurring job execution.
- Backups, restore tests, monitoring, alerting, and documented incident response.
- Search results should be paginated and indexed; large analytics queries should not block critical rent/payment operations.
- All dates stored consistently (UTC timestamps for events, explicit local date/time rules for rent due dates); currency stored with each charge.
- Audit records for important status transitions, document reviews, rent adjustments, and manual payment confirmation.

18. Acceptance Criteria for the First Release

34. An owner can create at least two properties and multiple units under each property.
35. A renter can search listings, open a listing, and contact the owner through at least one working channel.

36. An owner can review an inquiry/application and create a tenancy for the selected unit.
37. One tenancy/unit can show multiple active tenants, and each tenant has a separate account and private document area.
38. An owner can view the rented unit and see the correct active tenants without seeing unrelated tenants or properties.
39. The system generates rent charges on the configured schedule and distinguishes due, paid, partial, failed, and overdue states.
40. A payment is not shown as confirmed until verified by provider event or reconciliation/authorised manual confirmation.
41. Owner analytics reconcile to the underlying charges and confirmed payments for the selected date range.
42. Tenants can create a request, add details/attachments, receive updates, and view its resolution history.
43. Unauthorised users cannot access private documents, payment data, tenant profiles, or owner dashboards by changing a URL or API identifier.

Additional acceptance criteria for the owner-based SaaS model:

- Admin can create an owner account, assign a subscription plan, and send a secure activation invitation.
- Each owner receives a unique working subdomain and sees only their own public listings on that site.
- Owner subscription status and invoices are visible separately from tenant rent payments.
- A domain can be verified and mapped to exactly one owner; unverified domains cannot serve owner content.
- Automated authorization tests prove that one owner cannot access another owner's private data.

19. Decisions to Confirm Before Development

- Launch market and legal setup: Germany only at first, or multiple countries?
- Property structure: whole homes only, or rooms/shared accommodation and individual units from day one?
- Rent responsibility: one payment per tenancy, or separate rent shares/payments for each co-tenant?
- Payments: bank transfer with reconciliation first, or an integrated payment provider from launch? Which currencies are required?
- Owner payout model: direct payment to each owner's account, or platform collection followed by payouts? This changes the legal and technical design.
- Documents: exactly which document types are needed, who reviews them, and how long they are retained?
- Rental agreement: upload an existing agreement, use an e-signature provider, or keep signing outside the app in the MVP?
- Contact policy: should phone/email/WhatsApp be visible immediately, or only after an inquiry/application?
- Will property managers/agents have delegated access in the first release?
- Existing database and codebase: continue the current MongoDB implementation or switch to PostgreSQL before the schema grows?

20. Suggested Build Order

Phase	Workstream	Outcome
1	Foundation and permissions	Authentication, roles, API structure, database schema, audit logging, environments.
2	Properties and listings	Owner portfolio, units, listing creation/editing, public search and details.
3	Inquiries and communication	Application statuses, owner contact actions, conversation threads, notifications.
4	Tenancy and documents	Tenancy creation, multiple tenants, invitations, private uploads, review workflow.
5	Rent ledger and payments	Recurring charge generation, bank transfer/provider flow, webhooks/reconciliation, receipts.
6	Requests and analytics	Maintenance tickets, owner dashboard, rent

		reports, overdue and occupancy metrics.
7	Hardening and launch	Security testing, permissions tests, payment edge cases, backups, monitoring, legal review, pilot feedback.

Recommended first vertical slice: create one listing → renter submits an application → owner accepts → two tenants are added → documents are uploaded → a rent charge is created → a payment is confirmed → the owner dashboard updates → a tenant raises a maintenance request. This tests the product's most important connected workflow.

Appendix: Plain-Language Product Summary

For owners: "Manage all your homes, see who lives in each unit, collect and track rent, securely manage tenant documents, and resolve tenant requests from one dashboard."

For renters: "Find a home, contact the owner, complete your move-in process, upload required documents, pay rent, and request help during your stay—all in one place."